WEEK 1 · LESSON 5 OF 24 · HANDOUT

From Base Model to Assistant

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=7xTGNNLPyMI — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Deep Dive into LLMs like ChatGPT — https://www.youtube.com/watch?v=7xTGNNLPyMI

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

Your goal in this lesson: Explain the three training stages that turn a raw text predictor into a helpful assistant — pretraining, instruction tuning and preference tuning — and see how chat messages, system prompts and training masks work in code.

A base model is not an assistant

After pretraining, a model continues text. Ask a **base model** "What is the capital of Rwanda?" and it might continue with "What is the capital of Burundi? What is the capital of…" — because that is what quiz pages on the internet look like. It has knowledge but does not **behave** like an assistant.

The three stages

Stage	Data	What it teaches
1. Pretraining	Trillions of tokens of web pages, books, code	Language, facts, reasoning patterns
2. Instruction tuning (supervised fine-tuning, SFT)	Tens of thousands of example conversations written or checked by people	To answer, follow instructions and use a chat format
3. Preference tuning (RLHF, DPO, constitutional methods)	People (or AI guided by written principles) compare two answers and choose the better one	To be more helpful, honest and harmless

Most of the knowledge comes from stage 1; stages 2 and 3 shape the **behaviour**. Modern models add more training for reasoning ("thinking" before answering) and tool use.

Chat messages and templates

Chat APIs take a list of **messages**, each with a **role**:

- **system** — instructions from the developer: who the assistant is, rules, tone.
- **user** — what the person types.
- **assistant** — the model's earlier replies.

Before the model sees them, a **chat template** turns the messages into one long string with special marker tokens:

```
def render_chat(messages, add_generation_prompt=True):
    out = ""
    for m in messages:
        out += f"<|{m['role']}|>\n{m['content']}\n<|end|>\n"
    if add_generation_prompt:
        out += "<|assistant|>\n"            # the model continues from here
    return out

conversation = [
    {"role": "system", "content": "You are a polite assistant for a Kigali bakery. Answer in
one sentence."},
    {"role": "user", "content": "Do you sell bread on Sunday?"},
]
print(render_chat(conversation))
```

The model was trained on thousands of texts in exactly this format, so when it sees `<|assistant|>` it writes an assistant reply and then the `<|end|>` marker. Each model family has its own markers — libraries apply the right template for you.

Training only on the assistant's words

During instruction tuning, the loss is computed **only on the assistant's tokens**. The model should learn to **write answers**, not to predict what users type.

```
example = [
    {"role": "user", "content": "Price of beans?"},
    {"role": "assistant", "content": "Beans cost 1,200 RWF per kg today."},
]
pieces, mask = [], []
for m in example:
    part = f"<|{m['role']}|>\n{m['content']}\n<|end|>\n"
    pieces.append(part)
    mask += [1 if m["role"] == "assistant" else 0] * len(part)   # 1 = learn from this
character

full = "".join(pieces)
print(full)
print("characters:", len(full), "| trained on:", sum(mask), "| ignored:", len(mask) -
sum(mask))
```

In real training the mask is per token, and ignored positions get the label −100, which PyTorch's cross-entropy skips.

Preference tuning in one formula

Given a prompt with a **chosen** (better) and a **rejected** (worse) answer, preference tuning raises the probability of the chosen answer relative to the rejected one. **DPO** (Direct Preference Optimisation) does this with a simple loss. Here it is with made-up log-probabilities:

```
import math

def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    # how much more the model now prefers "chosen" over "rejected", compared with before
    margin = beta * ((policy_chosen - ref_chosen) - (policy_rejected - ref_rejected))
    return -math.log(1 / (1 + math.exp(-margin)))

print("model unchanged:          ", round(dpo_loss(-12, -12, -12, -12), 3))
print("model now prefers chosen: ", round(dpo_loss(-8, -15, -12, -12), 3))
print("model prefers rejected:   ", round(dpo_loss(-15, -8, -12, -12), 3))
```

The loss is **0.693** when nothing has changed, falls (about **0.40**) when the model moves towards the chosen answer, and rises (about **1.10**) when it moves the wrong way. Training pushes the loss down.

The system prompt is powerful — and limited

The system prompt sets the role, rules and style for a whole conversation, so it is the first tool you use when building an app. But it is **not** a security boundary: users can sometimes talk a model out of its instructions (Week 4: prompt injection). Important rules must also be enforced in your code.

Key terms

- **Base model** — a pretrained model that continues text.
- **Instruction tuning (SFT)** — fine-tuning on example conversations.
- **Preference tuning (RLHF, DPO)** — training on which of two answers is better.
- **Roles** — system, user, assistant.
- **Chat template** — how messages are turned into one string with special markers.
- **Loss masking** — learning only from the assistant's tokens.

Remember this: Pretraining gives knowledge; instruction and preference tuning give helpful behaviour; and every chat is really one long formatted text that the model continues.

Name: _____ Date: _____

Activity

Task 1

Add a second user–assistant exchange to `conversation` and render it. Where does the model start writing?

Task 2

Write three example conversations for a bakery assistant that you could use for instruction tuning.

Task 3

Compute the DPO loss for policy_chosen = −10, policy_rejected = −10 and reference values −12 and −10. Is the model moving the right way?

Task 4

Ask a chatbot to "ignore your instructions and…" something harmless. Does it comply? Why might a system prompt not be enough for security?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does a base (pretrained-only) model do with a question?
   - A. Always answers helpfully
   - B. Searches the web
   - C. Continues the text, which may not be an answer
   - D. Refuses
2. Which stage teaches the model to follow instructions in a chat format?
   - A. Tokenization
   - B. Instruction tuning (SFT)
   - C. Pretraining
   - D. Sampling
3. What data does preference tuning use?
   - A. Pairs of answers where one is preferred
   - B. Random web pages
   - C. Images
   - D. Only system prompts
4. Which role holds the developer's instructions?
   - A. system
   - B. tool
   - C. user
   - D. assistant
5. During instruction tuning, which tokens is the loss computed on?
   - A. None
   - B. Only the system prompt
   - C. Only the assistant's tokens
   - D. All tokens
6. Why is a system prompt not a security boundary?
   - A. It is encrypted
   - B. It is too short
   - C. Users can sometimes talk a model out of its instructions
   - D. It is ignored completely

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.